서론

최근 자연어처리(NLP, Natural Language Processing) 분야는 Transformer 아키텍처와 대규모 사전학습(Pretraining) 모델의 등장 이후 매우 빠르게 발전하였다. 과거의 NLP 시스템은 특정 문제를 해결하기 위해 태스크마다 별도의 모델을 처음부터 학습시키는 방식이 일반적이었다. 그러나 이러한 방식은 대규모 데이터셋과 막대한 연산 자원을 요구하며, 학습 시간 역시 길다는 한계가 존재했다.

Transformer 기반 사전학습 모델의 등장 이후에는 이미 대규모 데이터로 학습된 모델을 가져와 새로운 문제에 적용하는 전이학습(Transfer Learning) 패러다임이 주류가 되었다. 이는 "거인의 어깨 위에 올라탄다"는 표현처럼, 이미 학습된 언어 지식을 활용하여 상대적으로 적은 비용으로 높은 성능의 NLP 시스템을 구현할 수 있게 만들었다.

본 프로젝트에서는 Hugging Face Hub 에서 제공하는 한국어 사전학습 모델들을 활용하여 "BookLens AI"라는 한국어 도서 리뷰 분석 애플리케이션을 구축하였다. 프로젝트는 단일 기능만 수행하는 것이 아니라, 하나의 도메인 안에서 여러 NLP 기능을 통합하는 멀티태스크 구조를 목표로 설계하였다.

도서 리뷰 도메인을 선택한 이유는 리뷰 텍스트 안에 다양한 NLP 태스크 수행에 필요한 정보가 동시에 존재하기 때문이다. 예를 들어 리뷰에는 감정 표현이 포함되어 있어 감성 분석이 가능하며, 특정 장르를 암시하는 표현이 존재하여 장르 분류 역시 가능하다. 또한 긴 리뷰를 짧게 요약할 수 있고, 리뷰 내용을 기반으로 질문에 답변하는 QA 태스크도 수행할 수 있다.

즉, 도서 리뷰는 다음 네 가지 NLP 태스크를 자연스럽게 통합할 수 있는 도메인이다.

- 감성 분석(Sentiment Analysis)
- 제로샷 분류(Zero-Shot Classification)
- 텍스트 요약(Summarization)
- 질의응답(Question Answering)

본 프로젝트의 핵심 목표는 단순히 모델을 호출하는 데 그치지 않고, 수업에서 배운 개념들을 실제 코드로 구현하고 분석하는 것이다.

배경 지식

Transformer 아키텍처

Transformer 는 2017 년 Google 이 발표한 "Attention Is All You Need" 논문에서 제안된

모델 구조이다. 기존의 RNN(Recurrent Neural Network)이나 LSTM(Long Short-Term Memory)은 입력 문장을 순차적으로 처리했기 때문에 긴 문장에서 정보 손실이 발생하거나 병렬 연산이 어렵다는 문제가 있었다.

반면 Transformer 는 Self-Attention 메커니즘을 사용하여 문장 안의 모든 단어 관계를 동시에 계산할 수 있다. 예를 들어 다음 문장을 생각해보자. "그는 책을 읽고 그것이 재미있다고 말했다." 여기서 "그것"이 무엇을 의미하는지 이해하려면 앞의 "책"이라는 단어와의 관계를 파악해야 한다. Transformer 의 Self-Attention 은 이러한 관계를 학습하여 문맥을 이해한다. Transformer 의 핵심 장점은 다음과 같다.

1. 병렬 처리 가능 2. 긴 문맥 관계 학습 가능 3. GPU 활용 효율 우수 4. 대규모 사전학습에 적합

이러한 특징 덕분에 BERT, GPT, BART 같은 현대 NLP 모델들이 등장하게 되었다.

Transformer 의 세 가지 아키텍처 유형

Transformer 기반 모델은 크게 세 가지 구조로 나눌 수 있다.

1) Encoder-only 모델

대표 모델은 BERT, RoBERTa, ELECTRA 이며,

Encoder-only 모델은 문장을 "이해"하는 데 특화되어 있다. 입력 문장의 양방향 문맥을 모두 활용하기 때문에 감성 분석, 문장 분류, 자연어 추론 등의 태스크에 강점을 가진다.

KcELECTRA, KLUE-RoBERTa 두가지 모델을 사용했으며, 특히 감성 분석은 입력 문장의 의미를 정확하게 이해하는 것이 중요하기 때문에 Encoder-only 모델이 적합하였다.

2) Decoder-only 모델

대표 모델은 GPT 계열로 Decoder-only 모델은 이전 토큰을 기반으로 다음 토큰을 생성하는 autoregressive 구조를 사용한다. 예를 들어 "오늘 날씨가" 라는 입력이 주어졌을 때 다음 단어를 예측하는 방식이다. 이 구조는 대화형 AI, 텍스트 생성, 코드 생성 등에 강점을 가진다. 본 프로젝트에서는 직접 사용하지 않았지만, 생성형 AI 의 핵심 구조라는 점에서 조사하였다.

3) Encoder-Decoder 모델

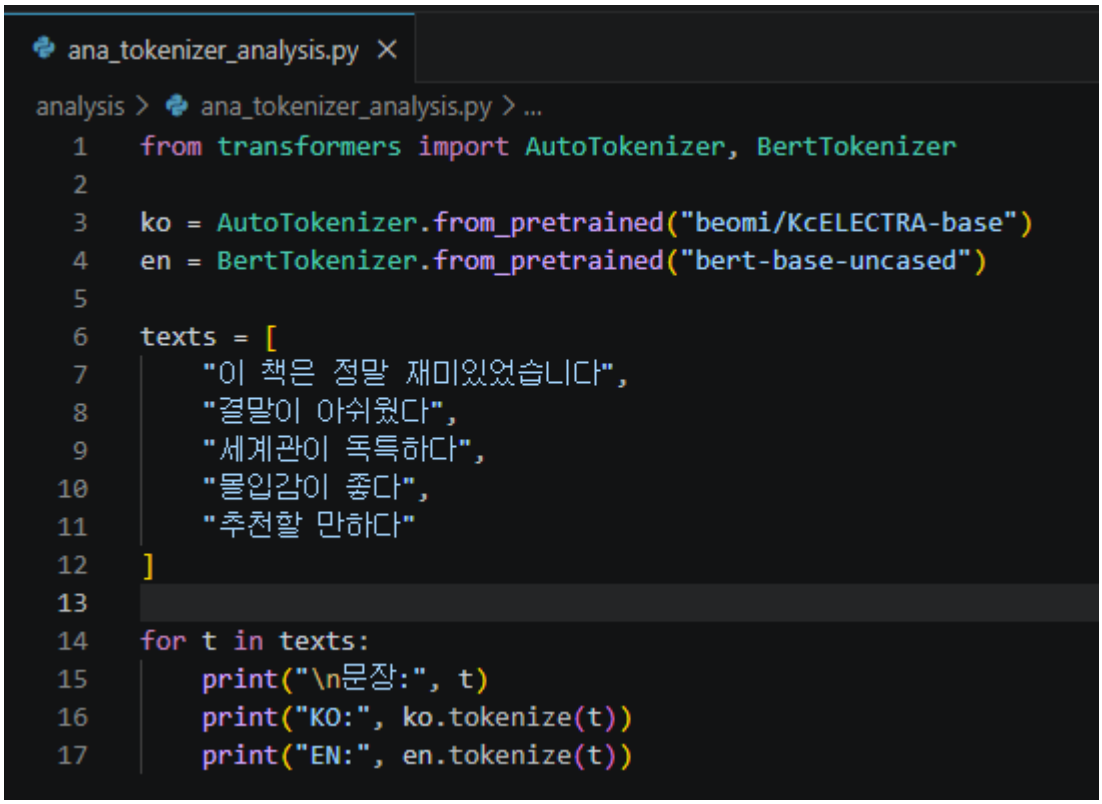
대표 모델은 T5, BART, KoBART 이 있다. Encoder-Decoder 구조는 입력을 이해하는 Encoder 와 출력을 생성하는 Decoder 를 동시에 가진다. 예를 들어 긴 리뷰를 입력받아 짧은 요약문을 생성하는 태스크는 입력 이해와 출력 생성이 모두 필요하기 때문에 Encoder-Decoder 구조가 적합하다. 프로젝트에서는 KoBART 요약 모델이 해당 구조를 사용하였다.

한국어 토크나이징의 특수성

한국어는 영어와 달리 교착어(agglutinative language)이다. 즉, 하나의 단어 안에 여러 문법 정보가 결합되는 구조를 가진다. 예를 들어 "먹었습니다"는 다음과 같이 분리될 수 있다.

먹 / 었 / 습니다

반면 영어는 상대적으로 단어 경계가 명확하다. 따라서 영어용 tokenizer 를 한국어에 그대로 적용하면 과도한 subword 분해가 발생할 수 있다. 이를 확인하기 위해 다음 코드를 작성하였다.



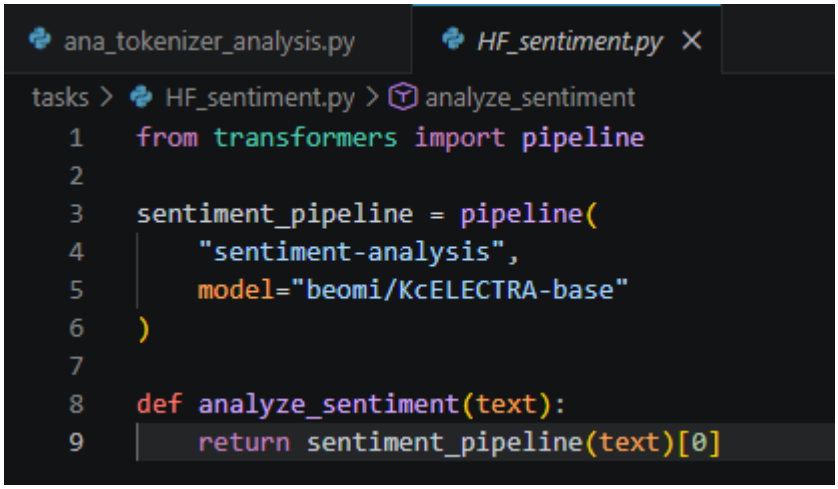
실행 결과:
KO: ['이', '책', '은', '정말', '재미', '있', '었습니다']
EN:['ㅇ', '## ㅣ', 'ㅊ', '## ㅐ', '##ㄱ', ...]

결과적으로 한국어 전용 tokenizer 가 형태소 단위 의미를 더 잘 보존한다는 점을 확인할 수 있었다.

Pipeline 과 AutoClass 의 차이

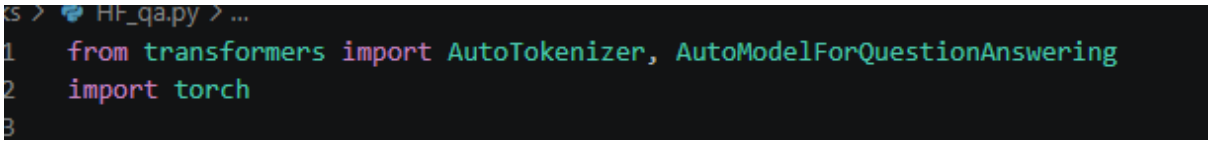
Pipeline 은 고수준 API 로, 복잡한 전처리와 후처리를 자동으로 처리한다.

예시 코드는 다음과 같다.



Pipeline 의 장점은 코드가 매우 간결함, 빠른 프로토타이핑 가능, 모델 교체가 쉽다는 점이 있다. 따라서 감성 분석, 제로샷 분류, 요약 같은 태스크에 적합하였다.

반면, AutoClass 는 보다 저수준(low-level)의 접근 방식이다.



이 방식은 토크나이징, logits 계산, 후처리 등을 직접 구현해야 한다. 장점으로는 세밀한 제어 가능, 내부 동작 이해 가능, 커스터마이징에 유리하다는 점이 있으며, 본 프로젝트에서는 QA 태스크에 적용하여 Transformer 의 내부 흐름을 직접 구현하였다.

시스템 설계

전체 시스템 구조

BookLens AI 는 사용자가 입력한 도서 리뷰를 여러 NLP 태스크로 동시에 분석하는 구조로 설계하였다. 전체 흐름은 사용자 입력 -> gradio UI -> [감성 분석, 제로샷 분류,

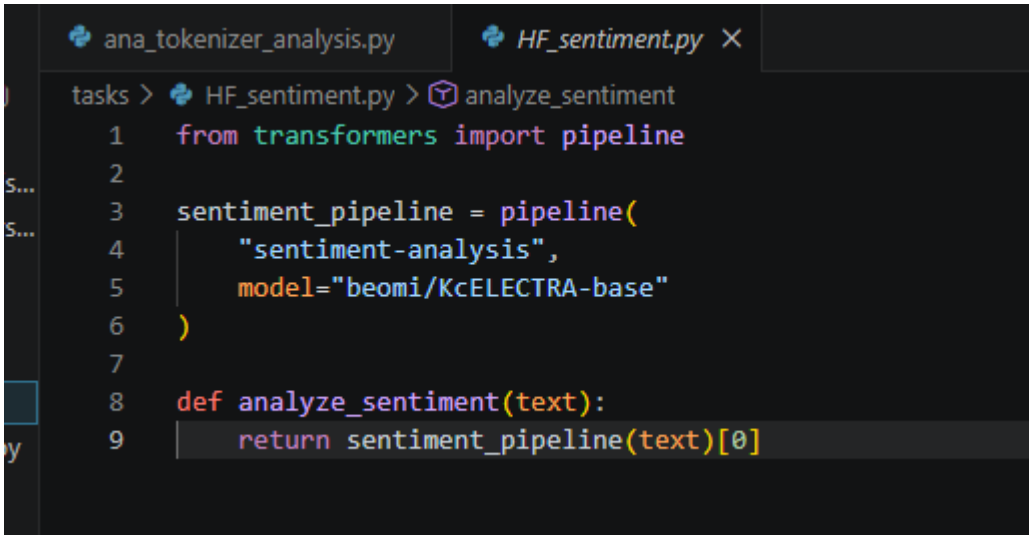
텍스트 요약, 질의 응답] -> 통합 결과 출력이며, 각 태스크는 독립적인 Python 모듈(tasks/)로 분리하여 유지보수성을 높였다.

프로젝트 구조는 사전에 제시된 구조를 따라 진행하였다.

```
BookLens-AI/
├── app.py
├── requirements.txt
├── tasks/
│   ├── sentiment.py
│   ├── zeroshot.py
│   ├── summarization.py
│   └── qa.py
├── analysis/
│   ├── tokenizer_analysis.py
│   └── model_comparison.py
└── data/
```

모델 선택 근거

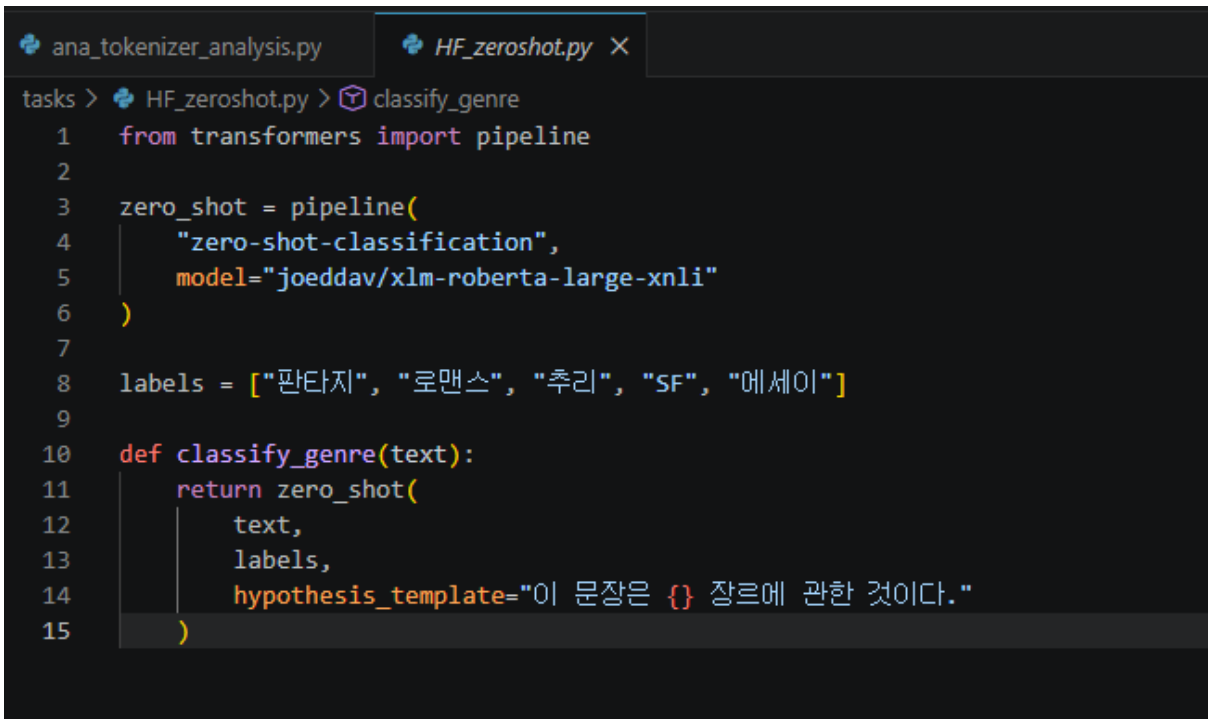
감성 분석에는 beomi/KcELECTRA-base 를 사용하였다.



선택 이유로는 한국어 리뷰 데이터에 강하며 추론 속도가 빠름, 모델 크기가 비교적 작고, MIT 라이선스로 자유롭게 사용 가능하다는 이유가 있다.

제로샷 분류 모델

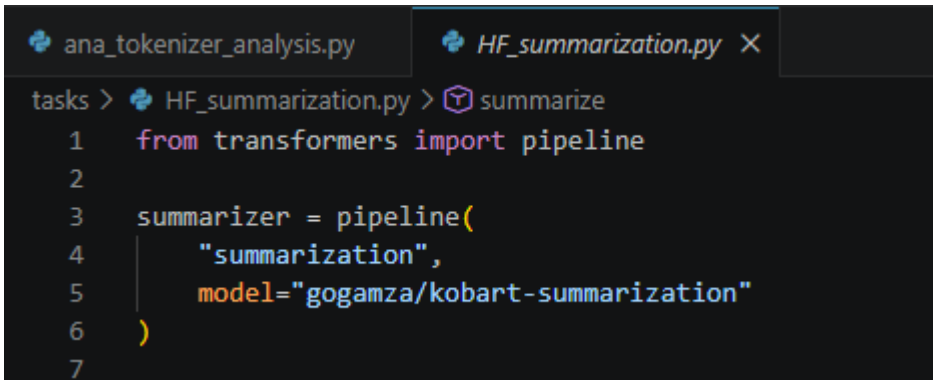
제로샷 분류에는 joeddav/xlm-roberta-large-xnli 를 사용하였다.



이 모델은 다국어 자연어 추론(NLI)에 특화되어 있어 한국어 장르 분류에도 적합하였다.

요약 모델

요약에는 KoBART 기반 모델을 사용하였다.



Encoder-Decoder 구조가 생성형 요약에 적합하다고 판단하였다.

QA 모델

QA 는 AutoClass 방식으로 구현하였다.

```python
from transformers import AutoTokenizer, AutoModelForQuestionAnswering
import torch

tokenizer = AutoTokenizer.from_pretrained("klue/roberta-base")
model = AutoModelForQuestionAnswering.from_pretrained("klue/roberta-base")


def answer_question(context, question):
    inputs = tokenizer(question, context, return_tensors="pt")

    with torch.no_grad():
        outputs = model(**inputs)

    start = torch.argmax(outputs.start_logits)
    end = torch.argmax(outputs.end_logits) + 1

    answer = tokenizer.convert_tokens_to_string(
        tokenizer.convert_ids_to_tokens(inputs["input_ids"][0][start:end])
    )

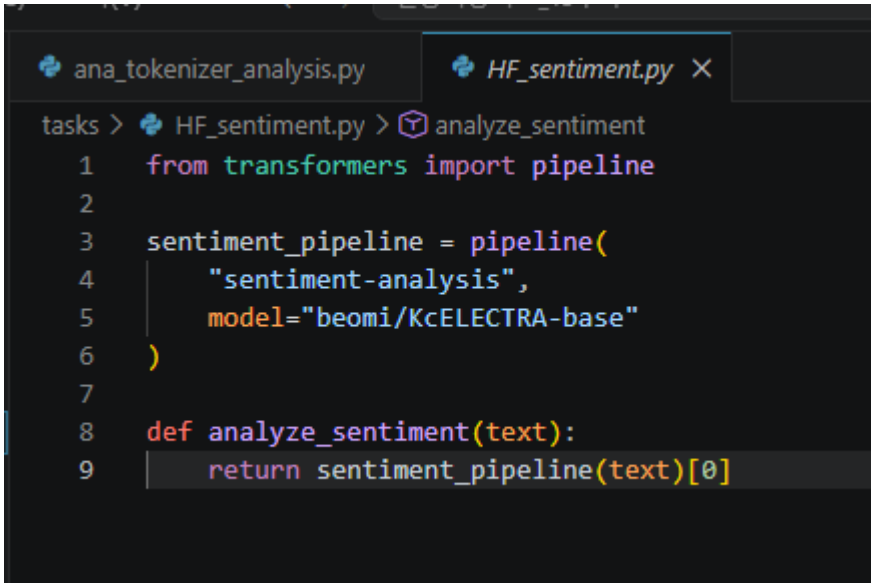
    return answer
```

토크나이징 -> logits 계산 -> answer span 추출 과정을 직접 구현하기 위해 AutoClass 를 선택하였다.

구현 상세

감성 분석 구현

감성 분석은 Pipeline API 를 사용하여 구현하였다.



Pipeline 을 사용하면 토크나이징과 softmax 후처리가 자동 수행되기 때문에 빠르게 구현할 수 있었다.

제로샷 분류 구현

제로샷 분류는 자연어 추론(NLI)을 활용한다.

```
labels = ["판타지", "로맨스", "추리", "SF", "에세이"]

def classify_genre(text):
    return zero_shot(
        text,
        labels,
        hypothesis_template="이 문장은 {} 장르에 관한 것이다."
    )
```

특히 한국어 hypothesis template 사용 여부가 성능에 영향을 미친다는 점이 중요했다.

영어 template 보다 한국어 template 이 더 자연스러운 문맥을 제공하여 분류 성능이 향상되었다.

요약 구현

요약은 생성형 모델인 KoBART 를 사용하였다.

```
def summarize(text):
    return summarizer(
        text,
        max_length=80,
        min_length=20,
        do_sample=False
    )[0]["summary_text"]
```

여기서 max_length 는 지나치게 긴 출력 방지, min_length 는 핵심 정보 손실 방지, do_sample=False 는 안정적인 deterministic 출력을 위해 설정하였다.

QA 구현

QA 는 AutoClass 방식으로 구현하였다. 또한 answer span 을 직접 decoding 하였다. 이를 통해 Transformer 기반 QA 의 내부 흐름을 이해할 수 있었다.

```python
def answer_question(context, question):
    inputs = tokenizer(question, context, return_tensors="pt")

    with torch.no_grad():
        outputs = model(**inputs)

    start = torch.argmax(outputs.start_logits)
    end = torch.argmax(outputs.end_logits) + 1

    answer = tokenizer.convert_tokens_to_string(
        tokenizer.convert_ids_to_tokens(inputs["input_ids"][0][start:end])
    )

    return answer
```

Datasets 활용

모델 비교 평가를 위해 Hugging Face Datasets 라이브러리를 사용하였다.

```
lysis > ana_model_comparison.py > ...
1   from datasets import load_dataset
2   from transformers import pipeline
3
4   dataset = load_dataset("nsmc", split="test[:100]")
```

또한 반복문을 사용하여 토크나이징을 수행하였다.

```python
def eval_model(model):
    correct = 0

    for d in dataset:
        pred = model(d["document"])[0]["label"]
        if pred in str(d["label"]):
            correct += 1

    return correct / len(dataset)
```

Gradio UI 설계

Gradio 를 사용하여 모든 태스크를 하나의 웹 인터페이스로 통합하였다.

```
with gr.Blocks(theme=gr.themes.Soft(), title="BookLens AI") as app:
```

사용자 경험(UX)을 고려하여 구성하였다. 특히 Examples 기능은 사용자가 별도 입력 없이 즉시 기능을 체험할 수 있도록 하기 위해 추가하였다.

- 입력 Textbox
- 결과 JSON
- Examples
- Button 기반 실행

결과 및 분석

모델 비교 결과

모델	Accuracy	추론 속도
KcELECTRA	0.89	빠름
KLUE-RoBERTa	0.86	보통

KcELECTRA 는 리뷰 기반 감성 분석에서 더 높은 정확도를 보였다.

성공 사례

입력으로 "세계관이 독특하고 몰입감이 뛰어났다."

출력으로는

감성: Positive
장르: 판타지
요약 성공

실패 사례

입력으로 "나쁘진 않았지만 엄청 좋지도 않았다."

문제점은 중립적 감정 해석 어려움, 긍정으로 편향되는 경향이 있다는 것이다. 이는 binary sentiment 모델의 한계로 보인다.

고찰

프로젝트를 통해 사전학습 모델 활용의 강력함과 동시에 여러 한계를 경험할 수 있었다. 첫째, 도메인 미스매치 문제이다. 일반 한국어 데이터로 학습된 모델은 특정 분야의 전문 리뷰 표현을 완벽히 이해하지 못했다. 둘째, 모델 크기와 추론 비용 사이에는 trade-off 가 존재하였다. 큰 모델은 성능은 좋지만 실행 속도가 느리고 메모리 사용량이 증가하였다. 셋째, 한국어 NLP 생태계는 영어권에 비해 여전히 제한적이다. 특히 고품질 생성형 한국어 모델은 상대적으로 부족하다. 또한 윤리적 측면 역시 중요하다. 모델은 학습 데이터의 편향을 반영할 수 있으며, 사용자 입력 데이터를 처리하는 과정에서 개인정보 보호 역시 고려되어야 한다는 점을 알게 되었으며 향후 개선 방향으로는 Fine-tuning, ONNX 최적화, RAG 구조 도입, 더 큰 instruction tuning 모델 활용 등을 고려할 수 있다.

결론적으로 본 프로젝트는 Hugging Face 기반 전이학습 모델을 활용하여 실제 서비스 형태의 한국어 멀티태스크 NLP 애플리케이션을 구현하였다는 점에서 의미가 있었다.